

Surrogate Model: GPR and 5-fold Grouped CV

A Surrogate ML Model for a Ti-6Al-4V Cementless Hip Implant

Recep Kamil Fırat
Computer Engineering · 2309011079

May 2026
ENS003 — Digital Twins for Health Sciences · Instructor: Prof. Dr. Şenol Pişkin

In this talk

- What the surrogate is for: pulling FEA output down to milliseconds
- Why Gaussian Process Regression (GPR), not XGBoost
- Cross-validation strategy: *why* 5-fold grouped
- R_{CV}^2 measured for all 10 targets
- Limits we report honestly

My responsibility (software team)

Surrogate model choice, hyperparameter tuning, cross-validation, `src/train.py` and `models/`.

The problem

The mechanical team solves each scenario with ANSYS Static Structural:

- One solve takes 5–10 minutes
- Covers a single mass/K/angle point
- **Unsuitable** for an interactive dashboard

FEA solve	5–10 min
Surrogate predict	≈ 1 ms
Speed-up	$\sim 10^5$

Surrogate goals:

- ≈ 1 ms response for the same inputs
- **Reliable interpolation** inside the training grid
- Posterior $\pm\sigma$ (calibrated uncertainty)

Model selection — 270 rows, target SF_{neck}

Head-to-head benchmark on the same data, 5-fold grouped CV:

Model	R_{CV}^2	MAE_{CV}	Unique outputs / 51
XGBoost (depth=1, stumps)	0.986	0.032	18
XGBoost (depth=3)	0.988	0.024	27
Kernel Ridge (RBF)	-0.005	0.369	5
Gaussian Process (RBF + white)	0.9999	0.00077	51

Smoothness probe: fix $K = 2.5, \theta = 0$, sweep the mass slider over 51 points. GPR produces a distinct value at every point (continuous curve); tree-based methods collapse onto few distinct values $\Rightarrow$ **stepped** function.

On 162 unique grid points, XGBoost averages ~ 6 samples per leaf, which is exactly what produces the step-function behaviour.

GPR architecture

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel, WhiteKernel

kernel = (
    ConstantKernel(1.0, (1e-3, 1e3))
    * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1e3))
    + WhiteKernel(noise_level=1e-3, noise_level_bounds=(1e-6, 1e-1))
)

base = GaussianProcessRegressor(
    kernel=kernel, normalize_y=True,
    n_restarts_optimizer=5, random_state=42, alpha=1e-8,
)

model = TransformedTargetRegressor(
    regressor=base, transformer=StandardScaler(),
)
```

- Kernel hyperparameters fit by marginal-likelihood maximisation
- `normalize_y` + `TransformedTargetRegressor` $\Rightarrow$ target-scale invariant
- Closed-form posterior $\Rightarrow$ every prediction comes with σ for free

CV strategy: *why* grouped?

Problem: 108 of 270 rows are ANSYS re-runs — the same (m, K, θ) point repeated with tiny numerical differences.

- With plain `KFold` / `LOO`, the same grid point can land in both train and test folds $\Rightarrow$ leakage
- Visible symptom: R^2 inflates artificially to ≈ 1.0
- We lose the ability to measure honest out-of-sample performance

Fix: `GroupKFold(n_splits=5)`, with the group ID coming from Ata's `grid_groups()` helper:

"m | K | theta" $\Rightarrow$ re-runs share a fold

The training loop

```
from sklearn.model_selection import GroupKFold

def _grouped_cv_eval(X, y, groups, n_splits):
    preds = np.full(len(y), np.nan)
    for tr, te in GroupKFold(n_splits).split(X, y, groups=groups):
        model, _ = _make_estimator()
        model.fit(X.iloc[tr], y.iloc[tr])
        preds[te] = model.predict(X.iloc[te])
    return {
        "r2_cv": r2_score(y, preds),
        "mae_cv": mean_absolute_error(y, preds),
    }
```

Per target:

- ① Out-of-fold predictions across 5 folds
- ② Final fit on the full 270 rows $\Rightarrow$ `models/<target>.joblib`
- ③ Metrics written to `models/metrics.json`

CV results — 10 targets

Target	R^2_{CV}	MAE _{CV}
safety_factor_neck_min	0.9999	7.7e-4
safety_factor_stem1_min	0.9999	8.3e-4
safety_factor_stem2_min	0.9997	4.4e-3
max_equivalent_vonmises_stress_Pa	0.9998	0.53 MPa
avg_equivalent_vonmises_stress_Pa	0.9998	10.6 kPa
min_equivalent_vonmises_stress_Pa	0.034	(FEA noise floor)
max_principal_stress_Pa	0.9998	0.57 MPa
avg_principal_stress_Pa	0.9998	5.0 kPa
min_principal_stress_Pa	0.9998	0.15 MPa
max_total_deformation_m	0.9998	7.4×10^{-7} m

9/10 targets clear $R^2 > 0.999$; min_equivalent sits at $\sim 10^{-10}$ — the FEA solver's noise floor. We report it honestly and hide it from the dashboard.

Honest limits

- **Interpolation is reliable, extrapolation is not.** Predictions for $m < 50$ kg, $K > 6$, $\theta > 20^\circ$ should be treated with care.
- **18-face fixed support is constant.** Different fixation patterns are not modelled; new FEA grid would be needed.
- $F_y = 0$ **in every row.** No out-of-plane loading; real activities are mostly in-plane so this is acceptable, but not complete.
- **Kernel hyperparameters near the bounds for some targets (ConvergenceWarning).** Marginal impact on R^2 ; the bounds can be widened later.

Fallback chain: GPR $\rightarrow$ XGBoost $\rightarrow$ sklearn-GBR — if sklearn's GPR is unavailable in an environment, the others kick in (`test_make_estimator_falls_back_to_sklearn_gbr`).

Demo

```
$ ./run.sh train
[...]
```

```
$ python -c "
import json
m = json.loads(open('models/metrics.json').read())
for r in m:
    print(f'{r[\"target\"]:42s}    R2_cv={r[\"r2_cv\"]:.4f}')
"
```

safety_factor_neck_min	R2_cv=0.9999
safety_factor_stem1_min	R2_cv=0.9999
safety_factor_stem2_min	R2_cv=0.9997

```
[...]
```

Test suite: 27 passed, 96% coverage. The dashboard reads `models/` and runs all 10 models per predict call.

Thank you.

Questions?

- `src/train.py` (GroupKFold + GPR + XGB fallback)
- `models/metrics.json` (live CV scores)
- `tests/test_train_predict.py`
- Development log: `docs/PROJECT_NOTES.md` Sprints 3–4